Generative mesh graph networks for imperfection-driven scatter in structural response fields

(authors)

Abstract

When some inputs of a manufactured structure are not measured, such as its geometric imperfections, one nominal design does not give one response. The same drawing gives a scatter of response fields, and a surrogate model has to reproduce that scatter, not only its mean. Field surrogates in computational mechanics are deterministic. Generative surrogates on meshes exist, but they reproduce the time variation of turbulent flows, and stochastic surrogates of input scatter predict scalar or low-dimensional outputs. We present HI-MGN-V, a hierarchical mesh graph network that generates complete displacement fields on unstructured finite-element (FE) meshes from the nominal design. A multilevel (V-cycle) graph network with 15 blocks is adjusted before every block by a random latent vector. During training the latent vector is inferred from the FE result; at prediction time it is drawn from a learned distribution conditioned on the design. The latent vector can be one vector for the whole structure (global), a field over coarse regions (local), or both (hybrid); the three options are three assumptions about how the unmeasured input acts on the response. We compare the predicted scatter with many FE analyses of the same nominal design on three criteria: (a) each generated field must look like an FE result, (b) the scatter at different locations must be correlated as in the FE results, and (c) the scatter must remain correct for designs outside the training range. The test problem is an axially compressed cylindrical shell with 64 imperfection samples per geometry, in which the same nominal shell either stays unbuckled or buckles. Earlier implementations of the global and local options fail in opposite ways: the local model has the right mean but too little scatter, and the global model underestimates the displacement and predicts too few buckled shells near the critical load. Results of the unified implementation are pending and are not reported here.

Keywords: response scatter, geometric imperfection, generative surrogate model, mesh graph network, shell buckling, flow matching

1. Introduction

1.1. Motivation

A finite-element model is deterministic once every input is given. In practice some inputs are never given. The as-built geometry of a thin shell differs from the drawing by imperfections of a fraction of the wall thickness. A fabricated part carries process variation, and a bolted joint has an unknown preload. We write ξ for such an input: a property of the individual part that affects the response but is not part of the nominal design and is not available to the analyst. A designer who works from the nominal design c therefore faces one design with many possible responses,

$$\mathbf{y} = S(c, \xi), \qquad \xi \sim p(\xi), \tag{1}$$

where S is the FE solver and $\mathbf{y}$ the response field. The quantity of interest is the conditional distribution $p(\mathbf{y} \mid c)$: the scatter of $\mathbf{y}$ obtained by running the solver over the distribution of ξ for a fixed design c.

Imperfection-sensitive structures make this scatter consequential. The axially compressed cylindrical shell is the classical case: its buckling load is controlled by imperfections that are small compared with the wall thickness, and the same shell can stay unbuckled or buckle [1]. A designer reads statistics of scalar quantities of interest (QoIs) from such a scatter, for example of the peak displacement magnitude $s = \max_i \|u_i\|$, the probability that the shell buckles, or the location of the buckle. Repeated FE analyses give these statistics at a cost of the number of designs times the number of imperfection samples per design. A surrogate that generates complete response fields from c and a random input removes the inner loop.

Three properties make the target hard to learn. A surrogate trained with a squared-error loss predicts the mean response, which for a shell near its critical load lies between the unbuckled and the buckled state and is not itself a possible response. Reproducing the scatter of each node separately does not ensure that distant regions deform together as in the FE results, and a QoI such as s depends on how they do. And a design study goes beyond the training data, so a surrogate has to be checked on designs it was not trained on, including designs near the critical load, where $p(\mathbf{y} \mid c)$ changes fastest.

1.2. Existing surrogates and the source of their scatter

Deterministic field surrogates. Most surrogates of field solutions in computational mechanics return one field per design. Projection-based and machine-learning-based reduced order models have been built for nonlinear aeroelasticity [2], parametric flow fields [3], fluid–structure interaction [4, 5], nonlinear parametric problems with a deep hierarchical variational autoencoder used as a compressor [6], and electromagnetic analysis of coated components [7]. Mesh graph networks treat the simulation mesh as a graph and exchange information between connected nodes [8, 9, 10]; multiscale variants reach distant nodes through coarsened meshes [11, 12, 13], and combined local and global attention has been used for the mechanical response of structures [14]. Operator networks map input functions to output functions [15, 16]. All of these return the mean response or a single field per design. They have no mechanism for ξ.

Generative surrogates of turbulent and chaotic flows. Generative models of physical fields exist, but nearly all of them reproduce the time variation of a turbulent or chaotic system. Latent diffusion graph networks generate states of fully developed flows on meshes [17]; scale-autoregressive generation [18] and graph flow matching [19] do the same; diffusion models generate turbulent states [20, 21, 22]; and ensemble weather models generate future atmospheric states [23, 24, 25]. In these problems the scatter comes from time: the reference data are the states of a long simulation, and every condition has many states by construction. A shell with an unmeasured imperfection is different. The solver is deterministic once ξ is given, the response is one static end state, and the scatter appears only because ξ is not known.

Surrogates of input scatter. Stochastic emulators model the scatter of a response as a function of the input, from repeated runs at the same input: generalized lambda models and stochastic polynomial chaos for scalar QoIs [26, 27, 28] and spectral surrogates for functional outputs [29]. In mechanics and design, conditional generative models have been applied to structural strain fields under random loads [30], to fabrication variation [31], to geometric uncertainty of metamaterials [32], and to imperfection-driven bifurcation [33, 34, 35]. These works already address parts of the problem studied here. Section 2.5 compares them property by property; none of them combines an unmeasured input, a high-dimensional field on an unstructured mesh, and a comparison with many FE results per design on the three criteria below.

1.3. Research gap

A generative surrogate of input scatter has to be compared with many FE results of the same design. We use three criteria.

(a) Realistic individual fields. Each generated field should look like one FE result, for example one clear buckle at one location rather than a blur of several possible buckles. A model can match the mean and the standard deviation at every node with fields that no FE analysis would produce.

(b) Spatial correlation of the scatter. The scatter at different nodes must be correlated as in the FE results: when the shell buckles at one location, the displacement elsewhere must change accordingly. A QoI such as the peak displacement depends on this correlation, and matching the scatter node by node does not ensure it.

(c) Designs outside the training range. The scatter must remain correct for designs the model was not trained on, including designs near the critical load, where the buckling probability changes fastest.

To our knowledge, no generative surrogate of structural response fields has been evaluated on all three (Table 1).

1.4. Contributions

1. **Problem definition and comparison with prior work.** We separate scatter caused by an unmeasured input from the time variation of turbulent flows and compare the closest prior work on the source of the scatter, the unmeasured input, the availability of several results per model input, and criteria (a)–(c) (Section 2, Table 1).
2. **HI-MGN-V.** A hierarchical mesh graph network that generates complete response fields on unstructured meshes. A random latent vector adjusts every block of a V-cycle graph network and is drawn from a conditional distribution that is fitted after the rest of the model is frozen. Where the latent vector acts (global, local or hybrid) is an assumption about how ξ acts on the field, and it is one switch in one implementation (Section 3).
3. **An evaluation protocol.** Many FE results per design as the reference, a posterior-versus-prior check that locates failures, statistics and calibration of scalar QoIs, scores for the spatial correlation, checks of

individual fields, and the position and shape of the buckle, on unseen, near-critical and out-of-range designs (Section 4.3).

4. **A shell-buckling dataset** with 64 imperfection samples for each of 150 geometries and 8 out-of-range geometries, in which the imperfection decides whether the shell buckles (Section 4.1).

Scope. The scatter studied here comes from an unmeasured input, not from time variation. The solver is deterministic once ξ is given, and the response is one static end state, not a time history. One dataset is used. The results of earlier implementations in Section 4.5 motivate the method; results of the unified implementation are pending.

2. Related work

2.1. Deterministic field surrogates

MeshGraphNets [10] encode a mesh as nodes and edges, process it with residual message-passing blocks, and map the node features to the outputs. Multiscale versions coarsen the mesh to pass information further per block [11]; bi-stride pooling builds the coarse levels from the connectivity [12]; and HI-MGN combines configurable coarsening with learned interpolation from neighbouring coarse nodes [13]. Patrignani and Pinho add combined local and global attention to graph networks for structural response [14]. Their global path passes information across the structure deterministically. A global random vector and a global information path are different mechanisms, and Section 3.5 keeps them apart. Reduced order models compress parametric solutions into a few coefficients [2, 3, 4, 6, 7, 5]; the variational autoencoder in [6] serves as a compressor and the surrogate is used deterministically. The graph network inside HI-MGN-V is the HI-MGN V-cycle.

2.2. Stochastic emulators of input scatter

Stochastic emulation treats the response at a given input as a random variable and learns its distribution from repeated runs. Generalized lambda models and stochastic polynomial chaos model the distribution of a scalar QoI [26, 27], including seismic structural response [28]. Spectral surrogates built from repeated time histories extend this to functional outputs [29]. These works establish the evaluation logic used here: repeated runs at the same input are the reference. They are not designed for fields on varying meshes with tens of thousands of nodes.

2.3. Generative surrogates of turbulent and chaotic flows

Latent diffusion graph networks (LDGN) generate states of fully developed flows from a compressed latent field on a coarse mesh [17]. They are trained on short simulations and compared, with a Wasserstein distance over the whole field, with the states of long simulations; their appendix also trains latent flow matching. Scale-autoregressive generation (SAR) generates the field coarse level first and refines it level by level [18], and PIFM-GN trains graph flow matching in physical and latent space [19]. Diffusion models generate turbulent states [20, 21] and flow uncertainty around airfoils [22]; GenCast, Graph-EFM and FGN generate ensemble weather forecasts [23, 24, 25]. Distributional operator learning with a diffusion last layer [36] or proper scoring rules [37] compares whole fields. These methods are the technical precedents for generating fields on meshes. Their scatter is the time variation of a turbulent state, so they neither model an unmeasured static input nor test against one.

2.4. Generative models of input scatter in mechanics and design

Several works are close to this paper and rule out a claim that the problem is untreated. Silionis et al. train a CVAE [38, 39] for strain fields of a plate and a ship hull under random loads [30]. The loads are not model inputs, so the model has to reproduce their effect as scatter; each FE run has its own thickness and load, and the reference statistics are pooled over the test set. Yang and Perdikaris train conditional implicit generators for stochastic systems [40], and engression trains a generator with a proper scoring rule [41, 42]. Gen-Fab predicts fabrication variation of nanophotonic devices and tests designs outside its training set [31]. GUST quantifies free-form geometric uncertainty of metamaterials from small data and passes generated shapes through a solver [32]. Han and Bahmani map imperfections to bifurcation outcomes with a conditional mixture model [33]. Equivariant flow matching generates alternative symmetry-breaking solutions, including buckled configurations [34], and Bi-FORK generates solution paths on distinct bifurcation branches [35]. Each of these covers some of the properties in Table 1. The present paper is the combination: an unmeasured input, a static field on an unstructured mesh, structural mechanics, and many FE results per design, compared on (a)–(c).

2.5. Positioning

Table 1 rates each line of work on six properties. *Source of scatter* is an input that the model does not receive (input), the states of a turbulent or chaotic simulation over time (time), or several solutions of one problem (bifurcation). Deterministic surrogates return one field per design and are left out. *Unobserved input* asks whether the model is trained without the variable that causes the scatter. *Several results per input* asks whether the data contain several solver results for the same model input, so that the predicted scatter can be compared with the FE scatter design by design instead of pooled over designs. This holds when the analysis is repeated with different values of the unobserved input, and when one long simulation provides many states for one condition (LDGN, SAR). Columns (a)–(c) are the three criteria of the research gap.

Work	Source of scatter	Unobs. input	Several results per input	(a) Realistic fields	(b) Spatial correl.	(c) Outside range
GLaM, SPCE [26, 27, 28]	input	O	O	–	X	X
Spectral surrogate [29]	input	O	O	X	△	X
CVAE, structural [30]	input	O	△	X	X	X
Implicit / scoring-rule [40, 41]	input	O	△	X	△	X
Gen-Fab [31]	input	O	△	△	X	O
GUST [32]	input	O	△	△	X	X
Han & Bahmani [33]	input	△	O	X	X	△
Equivariant FM [34]	bifurcation	O	O	△	△	X
Bi-FORK [35]	bifurcation	O	O	△	△	△
LDGN [17]	time	X	O	△	O	△
SAR [18]	time	X	O	△	O	△
PIFM-GN [19]	time	X	O	△	△	△
Turbulence diffusion [20, 21, 22]	time	X	O	△	O	△
Weather ensembles [23, 24, 25]	time	X	X	X	O	X
Distributional operators [36, 37]	mixed	△	O	X	O	X
This work	input	O	O	P	P	O

Table 1: Positioning. O: yes; △: partly; X: no; –: not applicable; P: planned in this paper, not yet computed. Unobs. input: the model does not receive the variable that causes the scatter. Several results per input: the data hold several solver results for one model input. (a) each generated field looks like a solver result; (b) the scatter at different locations is correlated as in the solver results; (c) designs outside the training range. Entries are our reading of each paper. For this work, (c) is evaluated on the earlier implementations only (Section 4.5).

2.6. Where the random variable enters the model

Generative models differ in where they put the latent variable. A *global* latent is one vector per sample: CVAEs, diffusion autoencoders [43], latent task-specific graph simulators [44] and a variational graph autoencoder for structural modal identification [45]. A *local* latent is attached to positions or regions: the coarse-mesh latent of LDGN, spatially varying modulation in image synthesis [46, 47, 48], and the per-scale latents of hierarchical segmentation models [49, 50]. *Hybrid* latents combine both: LION [51], PointFlow [52] and DualDiff [53]. The only controlled comparison of global, local and hierarchical latents we found is in image segmentation (HPU-Net, its Appendix F) [50]. Neither option is ruled out by theory: a global vector can carry a location through an input-dependent expansion [36], and local multiscale computation can reproduce long-range correlation [54]. Two further findings shape the design. A deterministic shortcut can bypass the random path [49], and feeding the same random vector to every layer can help or fail depending on the design [46, 55, 48].

2.7. Latent regularization and evaluation scores

The β-VAE trades reconstruction accuracy for agreement with the prior through a KL weight [56]. InfoVAE and the Wasserstein autoencoder replace the per-sample KL with a distance between the distribution of all latent vectors and the prior, often an MMD [57, 58, 59]. Regularized autoencoders and the two-stage VAE fit a distribution to the learned latent vectors after training [60, 61], as do diffusion autoencoders [43]; LSGM shows that training the prior jointly can matter [62]. Proper scoring rules are the basis for comparing a predicted distribution with observations [63]. The energy score is weak at detecting errors in correlation, and variogram scores complement it [64, 65]. Rank histograms show whether the scatter is too wide or too narrow [66], and precision, recall, density and coverage separate whether individual samples are realistic from whether the samples cover the reference scatter [67, 68]. With a finite number of FE runs per design, even a perfect model does not score zero [29, 69].

3. Method: HI-MGN-V

3.1. Problem statement

A sample is the FE mesh, treated as a graph $G = (V, E)$ with $N = |V|$ nodes and the element edges as graph edges. The nominal input c consists

of the reference coordinates, the shell thickness t, the applied end shortening Δ (constant over nodes, stored per node) and a node type (interior, clamped rim, loaded rim). The solver returns the displacement field $\mathbf{y} \in \mathbb{R}^{N\times 3}$ at the end of loading. The unmeasured input ξ enters the solver's initial geometry but not c. In the dataset of Section 4.1, ξ is one Gaussian dimple with three random variables: its amplitude, its angular position and its axial position. The training data are M geometries with K imperfection samples each, $\{(c_m, \mathbf{y}_{m,k})\}_{k=1}^{K}$, and the imperfections $\xi_{m,k}$ are not given to the model. The goal is a generator

$$\hat{\mathbf{y}} = D(c, \mathbf{z}), \qquad \mathbf{z} \sim p_\theta(\mathbf{z} \mid c), \tag{2}$$

whose output scatter matches $p(\mathbf{y} \mid c)$ on geometries not used in training and on geometries outside the training range. The latent vector $\mathbf{z}$ is not an estimate of ξ: it only has to describe the part of the effect of ξ that is visible in the response field.

3.2. Overview

Figure 1 shows the model. It has three parts. A multilevel (V-cycle) graph network D maps the nominal input and a latent vector to a displacement field. During training, an encoder $q_\phi(\mathbf{z} \mid c, \mathbf{y})$ infers the latent vector from the FE result. At prediction time the FE result is not available, and the latent vector is drawn from a learned distribution $p_\theta(\mathbf{z} \mid c)$ that depends only on the nominal input. In the usual VAE terminology, q_ϕ is the posterior and p_θ the prior. The placement setting (global, local or hybrid) changes the size of $\mathbf{z}$, which part of the encoder and of the learned distribution produces it, and which part of $\mathbf{z}$ each node reads. Nothing else changes.

3.3. Graph inputs and hierarchy

Each node carries [state | conditions | position | node type]: the state block (three displacement components, zero in this static, single-step problem), the thickness and end shortening (t, Δ), four position features and a three-way node-type indicator (interior, clamped rim, loaded rim). Each edge (i, j) carries eight features, the relative position in the current and in the reference configuration with their norms, $[\,\mathbf{d}_{ij}^{\mathrm{cur}}, \|\mathbf{d}_{ij}^{\mathrm{cur}}\|, \mathbf{d}_{ij}^{\mathrm{ref}}, \|\mathbf{d}_{ij}^{\mathrm{ref}}\|\,]$. In the static case the two coincide. Every input and output component is scaled to zero mean and unit variance with statistics of the training set.

The hierarchy has $L = 2$ coarse levels with 4096 and 256 clusters. Seed nodes are spread evenly over the mesh (farthest-point sampling), and every

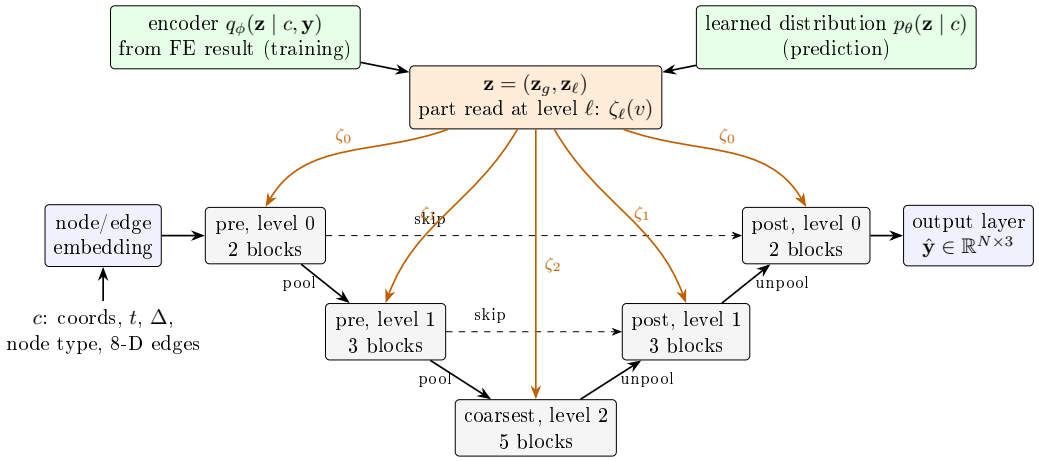


Figure 1: HI-MGN-V. The graph network is a 15-block V-cycle over three levels of one mesh (fine mesh, 4096 clusters, 256 clusters). Before every block, the node features are scaled and shifted by the part $\zeta_\ell(v)$ of the latent vector that the placement assigns to that node at that level (Table 2). During training the latent vector is inferred from the FE result $\mathbf{y}$; at prediction time it is drawn from the learned distribution, which sees only the nominal input. The graph network never sees $\mathbf{y}$.

node is assigned to the nearest seed along the mesh edges. A coarse node averages its members, and two coarse nodes are connected when their clusters touch. Each fine node v has an ancestor $a_\ell(v)$ at level ℓ and a *coarsest ancestor* $a_L(v)$. Two partitions with different random seeds are prepared per training sample and one is drawn per visit; inference uses a fixed seed. The partition is random, not a decomposition into physical regions.

3.4. V-cycle graph network

Node and edge features are mapped to vectors of width $H = 128$. The network has 15 blocks distributed as $(2, 3, 5, 3, 2)$ over the fine level, level 1, level 2, level 1 and the fine level. In each block every edge is updated from its two end nodes and every node from its edges, and the result is added to the input of the block (message passing [10]). Going down, member features are averaged into the next coarser level. Going up, a fine node interpolates from neighbouring coarse nodes with learned weights and adds the features it had at the same level on the way down [13]. A small fully connected network maps the final fine-level features to the three displacement components. Without the latent vector, this network is the deterministic HI-MGN.

3.5. Latent placement: three assumptions about how ξ acts

Where the latent vector acts states what the model assumes about the effect of ξ on the field.

- *Global.* The effect of ξ is summarized by a few variables shared by the whole structure, for example whether the shell buckles and how large the deformation is. The latent vector is one vector per level, read by every node.
- *Local.* The effect of ξ is regional: an imperfection at one place changes the response nearby. The latent vector is a field on the coarsest clusters, so a location can be written directly into it.
- *Hybrid.* Both: a shared variable and a regional field, sampled jointly.

In the shell problem ξ has only three random variables, but its effect is both global (the buckled or not) and localized (where the dimple, and hence the buckle, sits). A low-dimensional ξ does not imply that a global vector suffices, and a localized effect does not imply that a local field suffices: a global vector can encode a location [36], and a local field acting on a multilevel network can

produce correlation over long distances [54]. We therefore treat the placement as a modelling choice and test it as an ablation (Section 4.4). A global vector also adds a path that passes information across the whole structure, as deterministic global attention does [14]; a control with a deterministic global vector separates the two effects.

Formally, the latent has up to two parts:

$$\mathbf{z}_g \in \mathbb{R}^{S\times D},\ S = L+1 = 3,\ D = 32; \qquad \mathbf{z}_\ell \in \mathbb{R}^{n_c\times C},\ n_c = 256,\ C = 4. \tag{3}$$

The global part holds one vector of length D per level, and the local part one vector of length C per coarsest cluster. The placement selects which parts exist and defines the part $\zeta_\ell(v)$ that node v reads at level ℓ (Table 2). Blocks at a level read that level's part, on the way down and on the way up.

Placement	Parts	$\zeta_\ell(v)$ read at level ℓ	$\dim\zeta$	size of $\mathbf{z}$
global	$\mathbf{z}_g$	$\mathbf{z}_g[\min(\ell, S-1)]$	32	96
local	$\mathbf{z}_\ell$	$\mathbf{z}_\ell[a_L(v)]$	4	1024
hybrid	$\mathbf{z}_g, \mathbf{z}_\ell$	$[\,\mathbf{z}_\ell[a_L(v)] \mid \mathbf{z}_g[\min(\ell, S-1)]\,]$	36	1120

Table 2: Part of the latent vector read by node v at level ℓ. In the local and hybrid placements every node reads the regional vector of its coarsest cluster at *every* level, so the regional vector acts on the whole V-cycle instead of only at the coarsest level.

3.6. How the latent vector acts on each block

Before each block B_j, the latent vector scales and shifts the node features and sets how much of the block update is kept. With h the node features entering the block and ζ the part of the latent vector read by the node,

$$(\gamma,\ \beta,\ g) = W_j\,\mathrm{SiLU}(\zeta) + b_j, \qquad \gamma,\beta,g \in \mathbb{R}^H, \tag{4}$$

$$\tilde{h} = h \odot (1+\gamma) + \beta, \tag{5}$$

$$h' = h + g \odot \big(B_j(\tilde{h}) - \tilde{h}\big). \tag{6}$$

W_j is initialized to zero and b_j to $(\gamma,\beta,g) = (0,0,1)$, so at initialization $h' = B_j(h)$: the network starts as the deterministic HI-MGN, and the latent vector has an effect only once training changes W_j. This feature-wise scale and shift is known as FiLM [70]; a similar form is used in diffusion transformers [71], there with a normalization step and a gate that starts at zero. It costs $15\times(\dim\zeta+1)\times 3H$ parameters: 190,080 (global), 28,800 (local) and 213,120 (hybrid).

3.7. Encoder: the latent vector from the FE result

During training the encoder infers the latent vector from the FE result. The scaled field $\mathbf{y}$ is appended to the node inputs and processed by three message-passing blocks on the fine mesh (the encoder trunk). Two output branches read the trunk.

- *Global branch.* A learned weighted average over the nodes of each geometry, followed by linear maps to the mean and log-variance $\mu_g, \log \sigma_g^2 \in \mathbb{R}^{S\times D}$.

- *Local branch.* Trunk features are averaged into the coarsest clusters. Two blocks run on the coarsest clusters, and linear maps give $\mu_\ell, \log \sigma_\ell^2 \in \mathbb{R}^{n_c\times C}$.

The standard deviation is bounded below by 0.05. The latent vector is drawn as $\mathbf{z} = \mu + \sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which lets the training gradient pass through the random draw.

3.8. Regularization of the latent vector: MMD or KL

A regularization term keeps the inferred latent vectors close to a standard normal distribution $\mathcal{N}(0, I)$, so that they can later be generated without the FE result. Two choices are implemented.

MMD over the batch. The maximum mean discrepancy (MMD) compares the set of inferred vectors $\{z_i\}_{i=1}^n$ in a batch with draws $\{e_j\}_{j=1}^n \sim \mathcal{N}(0, I)$; it is zero when the two sets have the same distribution. We use the estimate that includes the $i = i'$ terms,

$$\widehat{\mathrm{MMD}}^2 = \frac{1}{n^2}\sum_{i,i'} k(z_i, z_{i'}) + \frac{1}{n^2}\sum_{j,j'} k(e_j, e_{j'}) - \frac{2}{n^2}\sum_{i,j} k(z_i, e_j), \qquad (7)$$

with a mixture of Gaussian kernels $k(a, b) = \frac{1}{5}\sum_m \exp\big(-\|a - b\|^2/(c_m h)\big)$, $c_m \in \{0.25, 0.5, 1, 2, 4\}$, and h the median squared cross distance (no gradient) [59, 57, 58]. Three properties of this estimate matter for the study.

- For $\mathbf{z}_g$ the estimate is computed for each level vector over the batch (two samples per GPU, collected over all GPUs) and averaged over the levels. Gradient accumulation does not enlarge this batch, and with so few samples the estimate stays positive on average even when the distributions match.

- For $\mathbf{z}_\ell$ every coarsest cluster of every graph is one row; at most 4096 rows are subsampled and the rows are not collected over GPUs. The rows of one geometry are correlated, so the term only matches the distribution of the four values of a cluster, pooled over clusters. It says nothing about the correlation between clusters, which the learned distribution of Section 3.9 has to reproduce.

- In the hybrid the two estimates are summed.

KL per sample. The Kullback–Leibler divergence of each sample's distribution $\mathcal{N}(\mu, \sigma^2)$ from $\mathcal{N}(0, I)$,

$$\mathcal{R}_{\mathrm{KL}} = \mathrm{mean}_{\mathrm{coords}}\Big[-\tfrac{1}{2}\big(1 + \log\sigma^2 - \mu^2 - \sigma^2\big)\Big], \tag{8}$$

is averaged over the components within each part and summed over the parts that exist. Its weight is 10^{-3}, against 1 for MMD.

3.9. Learned distribution of the latent vector (flow matching)

At prediction time the latent vector is drawn from a distribution conditioned on the nominal input. It is learned by flow matching [72, 73]: a network v_θ learns a velocity field that moves Gaussian random numbers ϵ to the latent vectors seen in training, along straight paths in a pseudo-time $t \in [0, 1]$. The target is the *scaled* latent vector $\tilde{\mathbf{z}} = (\mathbf{z} - m)/s$ (Section 3.10). With $t \sim U[0, 1]$ and $\sigma_{\min} = 10^{-4}$,

$$\mathbf{z}_t = \big(1-(1-\sigma_{\min})t\big)\epsilon + t\,\tilde{\mathbf{z}}, \qquad \mathcal{L}_{\mathrm{FM}} = \mathbb{E}\,\big\|v_\theta(\mathbf{z}_t, t, c) - \big(\tilde{\mathbf{z}} - (1-\sigma_{\min})\epsilon\big)\big\|^2. \tag{9}$$

A sample is obtained by integrating $\dot{\mathbf{z}} = v_\theta$ from $t = 0$ to 1 with 30 steps of Heun's second-order method [74] and undoing the scaling. The pseudo-time enters as sines and cosines of 16 frequencies $2^k\pi$, $k = 0, \ldots, 15$ (32 values). Each placement has its own network that reads the nominal input: three message-passing blocks on the fine mesh, without $\mathbf{y}$ (the condition trunk).

Global placement. The condition trunk is reduced by a learned weighted average to one vector κ per geometry. The velocity of the 96 components of $\mathbf{z}_g$ is a fully connected network of $[\mathrm{vec}(\mathbf{z}_t) \mid \tau(t) \mid \kappa]$.

Local placement. The condition-trunk features are averaged into the coarsest clusters, κ_v for $v \in V_L$. The node stream starts as $x_v = \mathrm{Lin}([\mathbf{z}_{\ell,t}(v) \mid \kappa_v])$ and passes through four blocks on the coarsest graph, each modulated by the time embedding with the gated form of Eq. (4). A linear head, initialized at 0.01 of its default scale, gives $v_\ell(v) \in \mathbb{R}^C$.

Hybrid placement. The hybrid adds one vector ω per geometry that carries the global part through the same integration:

$$\omega \leftarrow \mathrm{Lin}\big([\mathrm{vec}(\mathbf{z}_{g,t}) \mid \mathrm{Pool}(\mathrm{trunk})]\big), \tag{10}$$

$$\text{before block } j: \quad \omega \leftarrow \omega + \mathrm{MLP}_j\big([\omega \mid \bar{x} \mid \tau(t)]\big), \qquad x_v \leftarrow x_v + \mathrm{Lin}_j(\omega), \tag{11}$$

$$v_g = \mathrm{Lin}([\omega \mid \bar{x}]), \tag{12}$$

where $\bar{x}$ is the mean of the node features over the clusters and Pool the learned weighted average. The maps Lin_j start at zero, so at initialization the regional part behaves as in the local placement. One pseudo-time per geometry drives both parts, and the loss is the sum of the regional and global flow-matching terms. The two parts are therefore generated together and exchange information at every velocity evaluation; neither is generated first, and they are not generated independently.

3.10. Training and sampling

Loss. With the predicted field $\hat{\mathbf{y}} = D(c, \mathbf{z})$ and $\mathbf{z}$ inferred from the FE result,

$$\mathcal{L} = \alpha\,\mathrm{MSE}(\hat{\mathbf{y}}, \mathbf{y}) + \lambda\,\mathcal{R} + \beta\,\mathcal{L}_{\mathrm{PV}} + w\,\mathcal{L}_{\mathrm{FM}}, \qquad \alpha = 1000,\ \beta = 10,\ w = 1, \tag{13}$$

with $(\mathcal{R}, \lambda) = (\mathrm{MMD}, 1)$ or $(\mathcal{R}_{\mathrm{KL}}, 10^{-3})$. $\mathcal{L}_{\mathrm{PV}}$ is the squared error of the per-graph peak-to-valley $\max_i \hat{y}_{i,z} - \min_i \hat{y}_{i,z}$ of the axial displacement, an additional term that weights an extreme value of the field. It is not the QoI that is scored: the axial peak-to-valley is set largely by the imposed shortening, whereas $s = \max_i \|u_i\|$ also contains the radial buckling displacement. The flow-matching term is trained on a separate draw from the encoder, with μ and σ held fixed, so it does not change the encoder. The loss is a weighted sum chosen for the field, not the likelihood bound of a standard VAE.

Two phases. Algorithm 1 summarizes training. In the joint phase (epochs 1–80 of 120) all parts train together. At epoch 80 the graph network and the encoder are frozen, and the scaling of the flow-matching target is computed from the latent vectors that the frozen encoder infers over the training set,

$$m = \mathbb{E}[\mu], \qquad s = \sqrt{\mathrm{Var}(\mu) + \mathbb{E}[\sigma^2]}, \tag{14}$$

that is, their mean and variance over the training set (per component for $\mathbf{z}_g$, per channel for $\mathbf{z}_\ell$). The learned distribution then continues alone with a

new optimizer and learning-rate schedule; the scaling values are also copied into the running average of its weights that is used for prediction. In this phase the weights are updated after every batch, without the gradient accumulation of the first phase. The saved model is chosen by the continuous ranked probability score (CRPS) of the peak displacement s on the validation geometries, from 8 generated fields per geometry.

Algorithm 1 Training HI-MGN-V (any placement)

for epoch $= 1, \dots, E_{\text{freeze}}$ **do** ▷ joint phase
  for batch $(c, \mathbf{y})$, partition drawn from the variants **do**
    $(\mu, \sigma) \leftarrow q_\phi(c, \mathbf{y})$; $\mathbf{z} \leftarrow \mu + \sigma \odot \epsilon$
    $\hat{\mathbf{y}} \leftarrow D(c, \mathbf{z})$ ▷ blocks scaled and shifted by $\zeta_\ell(v)$
    $\mathcal{L} \leftarrow \alpha\,\mathrm{MSE} + \lambda\mathcal{R} + \beta\mathcal{L}_{\text{PV}} + w\,\mathcal{L}_{\text{FM}}(\mathrm{sg}[\mathbf{z}'])$ ▷ $\mathbf{z}'$: second encoder draw, held fixed
    update all parameters
  end for
end for
freeze D, q_ϕ; compute (m, s) over the training set; new optimizer
for epoch $= E_{\text{freeze}} + 1, \dots, E$ **do** ▷ second phase
  update the learned distribution on $\mathcal{L}_{\text{FM}}$ of scaled encoder draws
end for

Prediction. For a new geometry: build the clusters with the fixed seed, draw ϵ, integrate the velocity field (30 Heun steps), undo the scaling, and run the graph network once. The integration acts on a 96-component vector (global) or on the 256 coarsest clusters (local and hybrid); the fine mesh is visited by the condition trunk once per geometry and by the graph network once per generated field. This is the main cost difference from generators that iterate on the fine mesh, such as SAR [18], where every denoising step is a fine-mesh network pass.

3.11. Parameter cost

Table 3 lists the parameter counts of the three MMD arms. The graph network differs only through the scale-and-shift layers. Most of the difference comes from the encoder branches and, above all, the learned distribution. The hybrid has 34% more parameters than the global arm, so a comparison of the default arms mixes placement with capacity; the study adds matched arms (Section 4.4).

Arm	total	graph network	of which scale/shift	encoder	distribution
global	4,141,989	2,924,035	190,080	640,449	577,505
local	4,924,431	2,762,755	28,800	948,488	1,213,188
hybrid	5,534,385	2,947,075	213,120	973,385	1,613,925

Table 3: Parameter counts of the default arms (MMD; the KL arms are identical). The arms are not capacity-matched.

4. Dataset and results

4.1. Dataset: axially compressed cylindrical shells

Figures 2–3 and Table 4 define the experiment. A thin cylinder is clamped at the bottom rim, and the top rim is pushed down by an end shortening Δ at a fixed speed. The classical buckling load is $P_{cr} = 3.803\,Et^2$ and the classical shortening $d_{cr} = 0.60523\,tL/R$; the load ratio is $\lambda = \Delta/d_{cr}$. Each geometry is solved 64 times, each time with a different Gaussian dimple in its initial shape. The model sees only the nominal, imperfection-free cylinder. Depending on the dimple, the same nominal shell stays unbuckled or buckles.

Buckling indicator. A draw is counted as buckled when $s = \max_i \|u_i\|$ exceeds 5.17 mm. The threshold lies between the largest unbuckled and the smallest buckled peak and agrees with the solver's buckling flag on 97.5% of the FEA draws.

Common recipes. The 64 imperfection recipes (amplitude, angle and axial quantile) are identical across all 158 geometries; the largest difference is 2.6×10^{-5} in the axial quantile. The test set therefore measures unseen shapes with seen recipes. Fresh-recipe FEA for the held-out geometries is part of the study (Table 6).

4.2. Evaluation sets

Figure 4 shows the three evaluation sets: 15 unseen test geometries inside the box, 8 near-transition geometries inside the box, and 8 geometries that each move one design variable outside the box. The out-of-range set is out of range in the raw design variables but mostly in range in the nondimensional groups that govern buckling (Table 5). Four of the eight geometries lie inside the training range of λ, R/t and L/R; the others leave it by at most 14% in one group, and only the $\Delta = 0.6$ mm geometry lies below the training range of λ. Criterion (c) is therefore tested mainly in the raw variables.

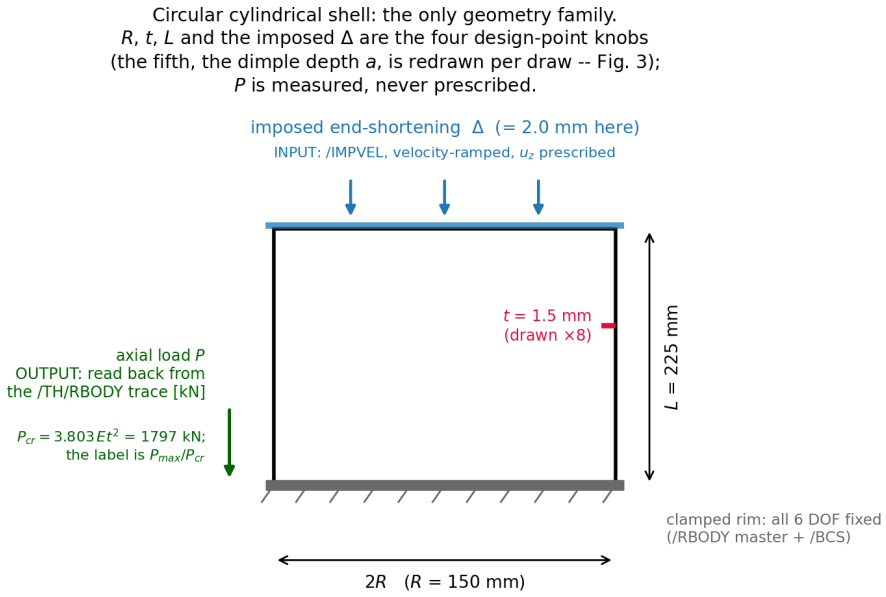


Figure 2: Boundary conditions, shown at the centre of the design box ($R = 150$, $t = 1.5$ (drawn $8\times$ thicker), $L = 225\,\mathrm{mm}$). The bottom rim is clamped in all six degrees of freedom. The top rim is driven axially by Δ at $0.25\,\mathrm{mm/ms}$. The reaction force is an output, not an input. $P_{cr} = 3.803\,Et^2$ is 1797 kN at the centre.

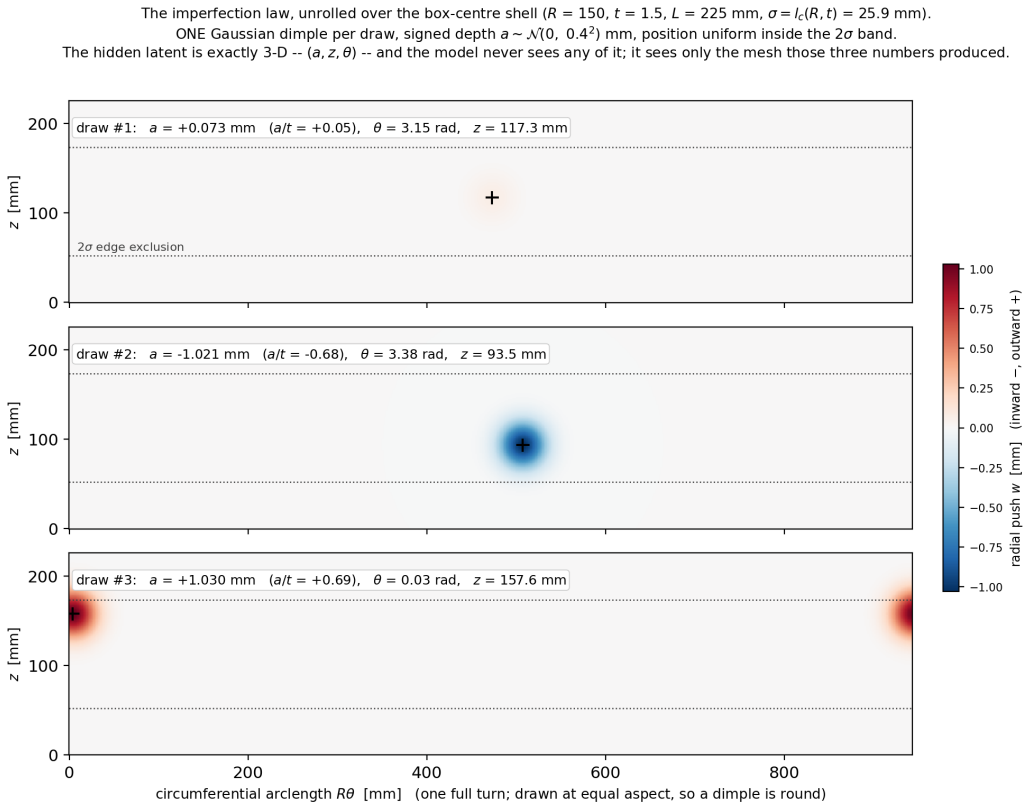


Figure 3: The unmeasured input ξ. Three samples on the unrolled centre shell ($a = +0.073,\ -1.021,\ +1.030\,\text{mm}$). The dimple width is $\sigma = l_c$ (25.9 mm at the centre). Its centre is uniform inside the dashed band, 2σ away from both rims. The imperfection enters only the solver's initial geometry; the model input is the nominal cylinder.

Item	Value
Design variables	$R \sim U[100, 200]$, $t \sim U[1, 2]$, $L \sim U[150, 300]$, $\Delta \sim U[1, 3]$ (mm); 4-D Latin hypercube, 150 geometries
Nondimensional range	λ 0.526–3.488; R/t 53.7–181.9; L/R 0.810–2.694
Imperfection ξ (not a model input)	64 samples per geometry; one Gaussian dimple ($K = 1$), three random variables: amplitude $a \sim \mathcal{N}(0, 0.4^2)$ mm, uniform angle, axial centre uniform away from a 2σ band at each rim; width $\sigma = l_c = \pi\sqrt{Rt}/[12(1-\nu^2)]^{1/4}$
Material	linear elastic steel, no yield; $E = 210\,\mathrm{GPa}$, $\nu = 0.3$, $\rho = 7.85 \times 10^{-6}\,\mathrm{kg/mm^3}$
Elements, mesh	structured quadrilateral shells, 5 through-thickness integration points; $h \leq l_c/10$; 27,975–87,398 nodes in training (median 45,409)
Solver	OpenRadioss explicit; units kg–mm–ms; terminal kinetic-to-internal energy ratio median 0.034, maximum 0.176 (descriptive, not an acceptance criterion)
Boundary conditions	bottom rim clamped (6 DOF); top rim axial velocity 0.25 mm/ms to Δ
HDF5 rows	0:3 coordinates; 3:6 displacement u; 6 thickness; 7 Δ; 8 node type
Split	by geometry (`split_group_attr shape_idx`, seed 42): 120 train / 15 validation / 15 test
Common recipes	all 158 geometries (training and out-of-range files) reuse the same 64 imperfection recipes

Table 4: Shell-buckling dataset. The imperfection parameters are stored for diagnosis only and are not model inputs.

shape	moved variable	λ	R/t	L/R	nodes	outside training range
150	$R = 80$	0.783	53.3	2.812	46,224	R/t (marginal), L/R
151	$R = 240$	2.350	160.0	0.938	45,982	–
152	$t = 0.8$	2.754	187.5	1.500	85,968	R/t
153	$t = 2.4$	0.918	62.5	1.500	28,718	–
154	$L = 120$	2.754	100.0	0.800	24,852	L/R (marginal)
155	$L = 350$	0.944	100.0	2.333	71,068	–
156	$\Delta = 0.6$	0.441	100.0	1.500	45,780	λ
157	$\Delta = 3.6$	2.644	100.0	1.500	45,780	–

Table 5: Out-of-range geometries in nondimensional groups. Training ranges: λ 0.526–3.488, R/t 53.7–181.9, L/R 0.810–2.694.

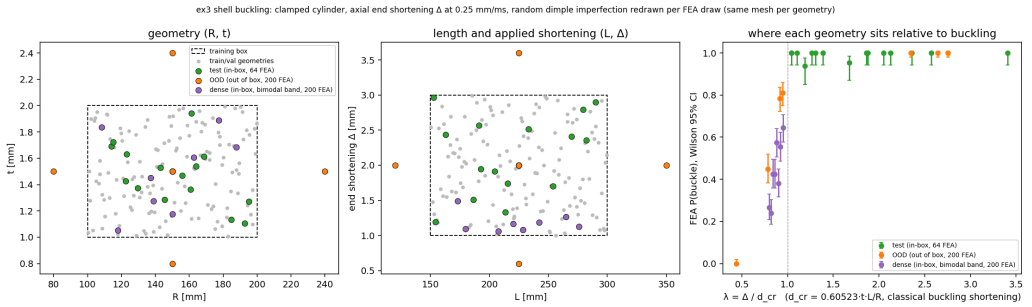


Figure 4: Evaluation sets. Test: 15 unseen geometries inside the box, 64 FEA draws each. Dense: 8 geometries inside the box near the transition ($\lambda \approx 0.80$–0.95). Out-of-range (OOD): 8 geometries that each move one variable from the box centre $(150, 1.5, 225, 2.0)$: R 80/240, t 0.8/2.4, L 120/350, Δ 0.6/3.6. Right: FEA buckling probability against λ with Wilson 95% intervals. The dense and OOD sets shown here have 200 draws per geometry; see the provenance note in Section 4.2.

Provenance of the evaluation draws. The released inference file contains the 8 out-of-range geometries with 64 draws each, generated with the same 64 recipes as the training set. The analysis of the earlier implementations in Section 4.5 used an earlier archive with 200 fresh draws per out-of-range and near-transition geometry. That archive, and the near-transition set as a whole, is not in the released files. The study either regenerates both sets with fresh recipes or documents the archive; until then, the out-of-range numbers of Section 4.5 cannot be reproduced from the released data.

4.3. Metrics

For each geometry, n_{model} generated fields are compared with the K FE results of the same geometry. Figure 5 shows why the comparison is made geometry by geometry: within one geometry the peak displacement has two clusters (unbuckled and buckled), and across geometries the buckling probability goes from 0 to 1 over $\lambda \approx 0.6$–1.05. The metrics are grouped by the three criteria of the research gap, after a check that locates failures and the scalar QoIs that a designer reads.

Posterior versus prior (run first). The graph network generates two sets of fields for each geometry: one from latent vectors inferred from the FE results (posterior), one from latent vectors drawn from the learned distribution (prior). If the first set matches the FE results and the second does not, the learned distribution is at fault; if both fail, the graph network or the encoder

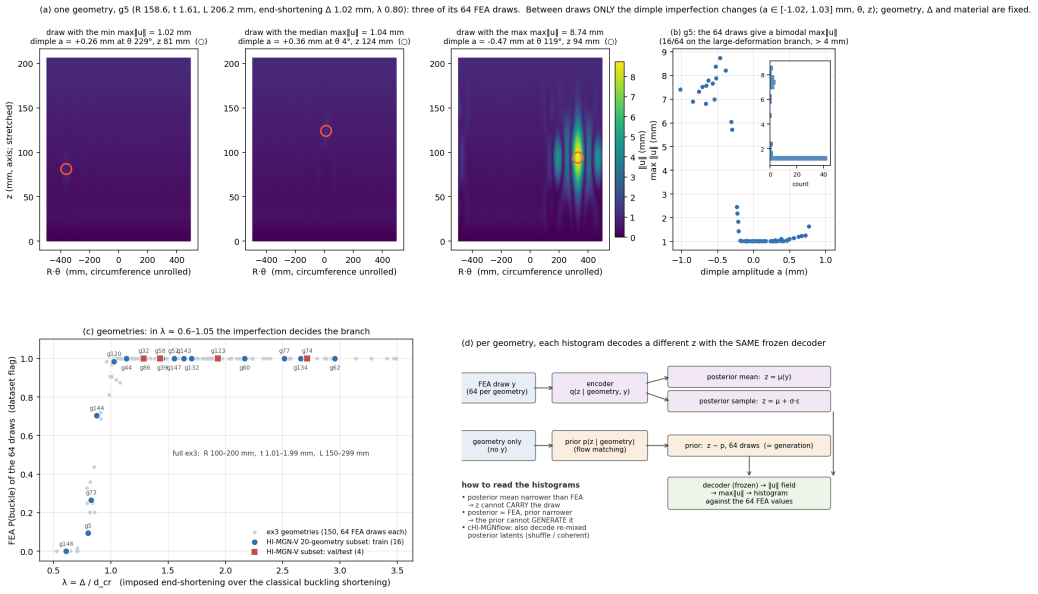


Figure 5: Diagnosis setup. (a) Three of the 64 FEA draws of geometry g5 ($\lambda = 0.80$); only the dimple changes between draws. (b) The 64 draws of g5 give a bimodal peak displacement. (c) FEA buckling probability against λ for all 150 geometries; the highlighted points are a 20-geometry subset used by an earlier run. (d) Posterior-versus-prior check: the same trained graph network generates fields from latent vectors inferred from the FE results (posterior mean and posterior draws) and from latent vectors drawn from the learned distribution (prior), and each set is compared with the FE results.

is. Two further checks mix the regional vectors of different FE results (shuffled) or keep all of them from one FE result (coherent). This check comes before any comparison of arms, because it tells which part of the model a difference belongs to.

Scalar QoIs. For the peak displacement $s = \max_i \|u_i\|$: the ratio of the means; the ratio of the standard deviations $\mathrm{sd}_{\mathrm{model}}/\mathrm{sd}_{\mathrm{FEA}}$ (spread ratio, 1 is correct, below 1 is too little scatter); the Wasserstein distance W_1 between the two histograms, which is the average horizontal distance between the two cumulative distribution curves; the CRPS, which measures the same distance against each FE result in turn; and the largest vertical gap between the two cumulative curves (Kolmogorov–Smirnov). W_1 and the CRPS are divided by $\mathrm{sd}_{\mathrm{FEA}}$. For the buckling probability: the error $|P_{\mathrm{model}} - P_{\mathrm{FEA}}|$ and a plot of predicted against FE probability over all geometries. Results are shown per geometry, as histograms of generated fields beside the FE results, not only as averaged ratios.

(a) Realistic individual fields. The generated fields and the FE results of a geometry are projected on the principal components of the FE results. Precision is the fraction of generated fields that lie close to at least one FE result; recall is the fraction of FE results that have a generated field close to them [67]. Density and coverage are the same two quantities computed in a way that is less sensitive to isolated outliers [68]. A model can match the mean and the scatter at every node with fields that no FE analysis would produce, for example a blur of two buckle positions; low precision detects this.

(b) Spatial correlation of the scatter. Two scores are computed on a fixed subset of nodes of each geometry (the mesh is the same for all FE results of a geometry). The energy score measures the distance between the generated and the FE fields with all nodes taken together. The variogram score compares, for every pair of nodes, the scatter of the difference between the two node values; it detects a model whose nodes scatter by the right amount but not together [64, 65].

(c) Designs outside the training range. Every metric above is reported separately on the test, near-transition and out-of-range sets of Section 4.2.

Buckle position and shape. For buckled fields: the axial and angular position of the peak, the direction of the peak displacement (inward or outward), and the dominant circumferential and axial wavelengths, each compared as a histogram with the FE results.

Reference value from the FE results alone. With a finite number K of FE results, even a perfect model does not score zero. Splitting the FE results of a geometry in half and scoring one half against the other gives the value that a perfect model would reach with the same number of samples [29, 69]. For W_1/sd its mean is 0.263, 0.189, 0.151 and 0.144 at $K = 16, 32, 64, 100$; for CRPS/sd it is about 0.52–0.54. The reference uses half the draws and is a comparison point, not a lower bound on what a model can score.

Statistics. Comparisons are paired on the same geometries, with geometry-level bootstrap intervals and at least three seeds. A difference between arms is reported as real only if it exceeds both the seed spread and the bootstrap interval on at least two of the three sets and does not come with a loss in criterion (a).

4.4. Baselines, arms and controls

Baselines. The deterministic HI-MGN gives the mean-response reference. Four generative baselines produce several fields per geometry: a CVAE on the same V-cycle with a Gaussian conditional distribution and a KL term [39]; a generator that adds random numbers to the deterministic HI-MGN and is trained with the energy score [41, 25]; an LDGN-type model that generates a compressed latent field on the coarse mesh by diffusion [17]; and SAR [18], run from the authors' released code adapted to the shell data. For SAR, each geometry is one SAR system and its 64 FE results take the place of the flow states. SAR receives the same inputs as HI-MGN-V (nominal coordinates, thickness, end shortening and node type) and generates the final displacement field; the imperfection parameters are not given. The SAR model, its two training stages (a graph VAE, then scale-by-scale flow matching over the frozen VAE), its mesh coarsening and its sampler are unchanged, and the hyperparameters are the authors' wing case. Only the data input, the training loop (plain PyTorch instead of the authors' training framework, with the same optimizer, learning-rate schedule and stopping rule), smaller batches with gradient accumulation to the authors' effective batch size, and a memory-saving but numerically equivalent form of the attention were changed. SAR runs a network at every mesh level and every

integration step, so its cost per generated field is higher than one pass of HI-MGN-V and is reported. PIFM-GN [19] is not included.

Arms. The six HI-MGN-V arms are {global, local, hybrid} × {MMD, KL}. They share every other setting (Appendix Appendix A): training on several GPUs with two samples per GPU and four accumulation steps in the first phase, learning rate 10^{-4}, weight decay 10^{-4}, 5 warm-up epochs, 120 epochs with the freeze at 80, mixed precision, a running weight average with factor 0.99, and geometry augmentation. The twelve configuration files (training and inference per arm) validate under the launcher, and the implementation passes 13 structural tests of shapes, routing and checkpoint round trips. These tests say nothing about accuracy.

Controls. The arms alone do not separate placement from model size, a random latent vector from a deterministic global information path, or where the latent vector enters from where it lives. Table 6 lists the controls.

4.5. Results

Results of the unified implementation. No arm of the unified implementation has been trained at the time of writing. Table 7 fixes the layout in which the results will be reported; every entry is pending.

Earlier implementations. The results below were obtained *before* the unification, with two separate implementations. "HI-MGN-V (global)" is the global placement on the same graph network. "HI-MGNflow (regional)" is a regional model with a different encoder (it reused the V-cycle features), a KL weight of 10^{-6}, a first training stage without the learned distribution, and a separately trained flow-matching distribution on the coarse clusters. The two differ in more than placement, and the out-of-range and near-transition sets come from the 200-draw archive described in Section 4.2. These numbers motivate the method; they are not results of it.

Observations.

1. *The two placements fail in opposite directions.* The regional model has the right mean (mean ratio 1.02–1.12) but too little scatter (spread ratio 0.49–0.66), and it predicts buckling where FE gives a mix (g150: 1.00 against 0.45). The global model underestimates the displacement (0.63–0.83). Its spread ratio above one on the test set comes from a long tail of unbuckled fields where FE buckles, not from a correct scatter

ID	Control	What it separates
A1	deterministic HI-MGN (MSE)	the value of any random latent vector; the mean-response reference
A2	size-matched arms	placement from parameter count: global vector widened toward ≈1024 components, local field narrowed, widths of the learned distributions matched
A3	attribution checks	hold $\mathbf{z}_g$ and redraw $\mathbf{z}_\ell$, and the reverse: which part carries which share of the scatter
A4	coarsest level only	the regional vector read at every level versus read once at level 2
A5	independent distributions	$p(\mathbf{z}_g \mid c)\, p(\mathbf{z}_\ell \mid c)$ instead of generating both parts together
A6	known imperfection	the three dimple variables given as inputs and sampled from their known distribution; the unmeasured input revealed
A7	deterministic global vector	a vector of the same width per geometry in the graph network, without randomness: random latent vector versus global information path
A8	$\lambda_{\mathrm{MMD}} = 0$	whether the regularization matters once the distribution is fitted after the freeze
A9	new-imperfection FEA	64 new imperfection samples for each held-out geometry: unseen shapes with unseen imperfections
A10	seeds	at least three training seeds per arm

Table 6: Controls. A1, A2, A6, A8 and A10 need only configuration changes or existing tools; A3–A5 and A7 need new code; A9 needs new FEA.

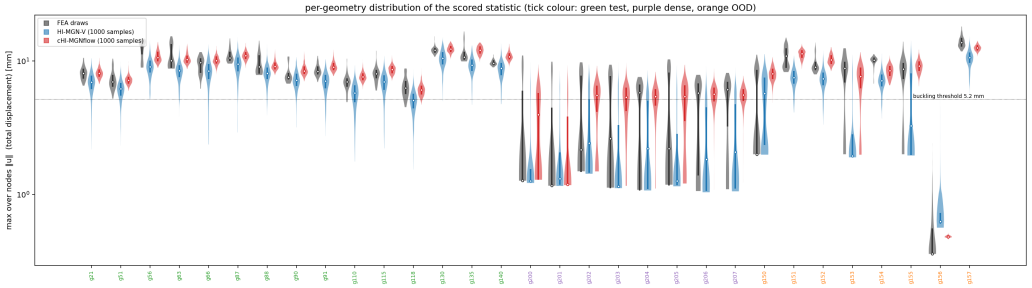


Figure 6: Earlier implementations. Per-geometry distributions of the peak displacement (log axis): FE (grey), the global implementation (blue, labelled HI-MGN-V) and the regional one (red, labelled with its former name cHI-MGNflow), 1000 samples each. Tick colour: green test, purple near-transition, orange OOD. The dotted line is the buckling threshold.

	peak displacement s				(b) correlation		(a) realism	
Model	mean	spread	W_1/sd	$\lvert\Delta P\rvert$	energy	vario.	prec.	recall
HI-MGN (deterministic)	–	–	–	–	–	–	–	–
CVAE	–	–	–	–	–	–	–	–
Energy-score generator	–	–	–	–	–	–	–	–
LDGN-type	–	–	–	–	–	–	–	–
SAR (ported)	–	–	–	–	–	–	–	–
HI-MGN-V global (MMD / KL)	–	–	–	–	–	–	–	–
HI-MGN-V local (MMD / KL)	–	–	–	–	–	–	–	–
HI-MGN-V hybrid (MMD / KL)	–	–	–	–	–	–	–	–

Table 7: Layout of the main results, one block per evaluation set (test, near-transition, out-of-range); the out-of-range block is criterion (c). mean, spread: ratios of model to FE mean and standard deviation of s; $\lvert\Delta P\rvert$: buckling-probability error; energy, vario.: energy and variogram scores; prec., recall: precision and recall. All entries are pending.

Set	Model	mean ratio	spread ratio	W_1/sd	$\lvert\Delta P\rvert$
test (15×64)	HI-MGNflow (regional)	1.02	0.66	0.69	0.005
	HI-MGN-V (global)	0.83	1.25	1.37	0.09
	FE vs FE			0.29	
OOD (8×200)	HI-MGNflow (regional)	1.07	0.49	0.83	0.10
	HI-MGN-V (global)	0.80	1.06	1.63	0.16
	FE vs FE			0.14	
dense (8×200)	HI-MGNflow (regional)	1.12	0.53	0.50	0.10
	HI-MGN-V (global)	0.63	0.66	0.56	0.28
	FE vs FE			0.11	

Table 8: Earlier implementations, peak displacement s per geometry (1000 generated fields per geometry, averaged over the set). The target for the ratios is 1. $\lvert\Delta P\rvert$ is the buckling-probability error. "FE vs FE" is the reference value from the FE results alone.

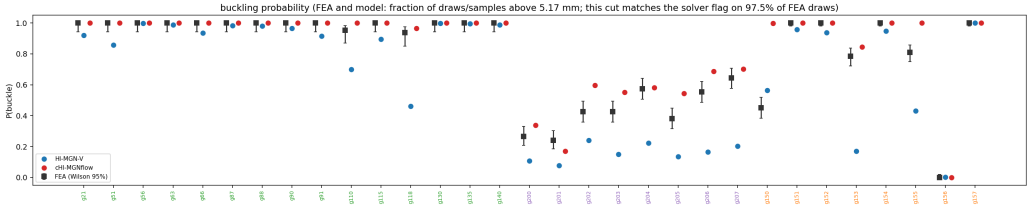


Figure 7: Earlier implementations. Buckling probability per geometry: FE with Wilson 95% intervals; models as the fraction of generated fields above 5.17 mm.

(Figure 6), and it predicts too few buckled shells near the transition (0.08–0.24 against FE 0.24–0.65 on the near-transition set).

2. *Where the error sits.* In the regional model, fields generated from latent vectors inferred from the FE results (posterior) reach a spread ratio of 0.97, fields from the learned distribution (prior) 0.57, shuffled regional vectors 0.48 and coherent ones 0.97: the learned distribution, not the graph network, loses the scatter. Making it four times deeper (4 to 16 blocks) moved the spread ratio only from 0.59 to 0.62, and the first three principal components of its fields carry 0.42–0.51 of the FE variance. In the global model, the amplitude of the fields generated from the posterior mean is 0.56–0.81 of the FE value and the latent vector explains about 47% of the scatter, so the global vector does not carry enough information to tell the 64 FE results of a geometry apart.
3. *The ranking is not the same on every geometry.* On geometry g150 the global model is far closer (W_1/sd 0.34 against 0.93). Later parameter studies moved both: a wider regional distribution network (width 128) removed the regional bias, and for the global model only a ten-fold MMD weight helped.

The pattern is the starting hypothesis for the placement statement of Section 3.5. A regional latent field reproduces where the response localizes but not how often the shell buckles; a global latent vector carries a shared variable but not enough spatial detail. Whether a hybrid, with both parts generated together, does better than either at the same model size is what the unified study tests.

5. Discussion and limitations

What each outcome would mean. If the hybrid beats both single placements at the same model size (A2) and loses that gain when its two parts are generated independently (A5), the gain comes from generating them together. If the deterministic global vector (A7) closes the gap to the hybrid, the global part helps by passing information across the structure, not by its randomness. If reading the regional vector at the coarsest level only (A4) works as well as reading it at every level, the every-level reading of Table 2 is unnecessary; if it fails, it is needed. If the model that is given the imperfection (A6) is far ahead of every arm, the difficulty is inferring the effect of ξ from the field, not the graph network. If no HI-MGN-V arm beats the energy-score

generator, the latent vector and its learned distribution are not needed for this problem.

Why the distribution is fitted after a freeze. The regional implementation showed that a distribution trained together with the encoder chases a moving target and ends with too little scatter. Fitting it to the latent vectors of the frozen encoder, after scaling, removes the moving target. LSGM reports the opposite trade-off for images [62]. The first phase still trains the distribution, so it starts the second phase from a good state, but its gradient does not change the encoder.

What the regularization can and cannot do. MMD makes the set of all inferred latent vectors look like a standard normal distribution, per level vector or per channel. It does not limit how much each vector says about its FE result, which suits a graph network that should use the latent vector. KL limits this for every sample and can make a weak latent vector carry no information [56]. In both cases the dependence on the nominal input is carried by the distribution fitted after the freeze. The six arms measure which regularization leaves latent vectors that this distribution can fit.

Limitations.

- No result of the unified implementation is reported. Section 4.5 uses earlier implementations that differ in more than placement.
- One dataset, with one dimple per analysis. Several interacting imperfections, material scatter or load scatter may favour another placement.
- All geometries, including the released out-of-range file, reuse the same 64 imperfection recipes, so the test measures unseen shapes with seen imperfections. The 200-sample archive behind the earlier out-of-range and near-transition numbers is not in the released files.
- The out-of-range set is out of range in raw design variables but mostly in range in the nondimensional groups (Table 5).
- The clusters are random. Regional latent vectors do not correspond to physical regions, and a model may depend on the node numbering through the first seed node.
- The default arms are not capacity-matched (Table 3).

- The global MMD is estimated on a batch of two samples per GPU, and the local MMD ignores the correlation between clusters (Section 3.8).
- The peak-to-valley term acts on the axial displacement, not on the scored QoI (Section 3.10).
- The objective is not a likelihood bound, so no likelihood comparison is possible.

Claims this paper does not make. Table 9 lists phrasings we avoid and why.

Not claimed	Reason / correct statement
"first to address one-to-many responses in mechanics"	stochastic emulators, CVAEs, Gen-Fab, GUST, Han & Bahmani, equivariant flow matching and Bi-FORK address parts of it (Table 1)
"first generative surrogate on meshes"	LDGN, SAR, PIFM-GN and Graph-EFM exist
"first hybrid latent"	LION, DualDiff, HPU-Net and others combine global and local latent variables
"first to evaluate (a)+(b)+(c)"	only after (a) and (b) are computed, and limited to structural fields with scatter from an unmeasured input
"the diffusion-transformer conditioning (AdaLN-Zero)"	there is no normalization and the gate starts at one: a gated FiLM
"ELBO" or "likelihood"	the loss is a weighted sum; MMD acts on the set of latent vectors, not on each one
"regional latent vectors are physical regions"	the clusters are random
"a global random vector helps"	a hypothesis, pending the deterministic-global control (A7)
"the method works outside the training range"	out-of-range geometries are *evaluated*; too little scatter and a biased mean were observed in the earlier implementations

Table 9: Wording constraints applied throughout.

6. Conclusion

When an input of the manufactured part is not measured, a structural surrogate has to generate the scatter of response fields, not one field. Existing field surrogates in mechanics are deterministic, generative surrogates on

meshes reproduce the time variation of turbulent flows, and surrogates of input scatter return scalar or low-dimensional outputs. HI-MGN-V generates complete response fields on unstructured FE meshes from the nominal design: a multilevel graph network whose blocks are adjusted by a latent vector drawn from a learned distribution conditioned on the design. Whether the latent vector acts globally, regionally or both is a stated assumption about how the unmeasured input acts, and it is one setting of one implementation. The evaluation compares generated fields with many FE analyses of the same design on three criteria: (a) each generated field looks like an FE result, (b) the scatter is correlated over the structure as in the FE results, and (c) both hold outside the training range. The test problem is a shell-buckling dataset in which the unmeasured imperfection decides whether the shell buckles. Earlier implementations of the global and local options fail in opposite ways on this dataset, which motivates the hybrid and the controlled study that will test it.

Appendix A. Hyperparameters

Group	Setting
Data	`input_var 3`, `output_var 3`, `cond_var 2`, `use_node_types`, `positional_features 4`, `edge_var 8`, `augment_geometry`, `split_group_attr shape_idx`, `split_seed 42`
Training	DDP; batch 2, `grad_accum_steps 4` (joint phase); lr 10^{-4}; weight decay 10^{-4}; warm-up 5; 120 epochs; AMP; checkpointing; EMA 0.99; `training_seed 42`; `best_by crps` with `vae_valid_prior_samples` 8 (default)
Graph network	`latent_dim 128`; `message_passing_num 15`; `use_multiscale`; `coarsening_type voronoi_seedmean`; `multiscale_levels 2`; `voronoi_clusters 4096, 256`; `mp_per_level 2,3,5,3,2`; `hierarchy_variants 2`; `hierarchy_seed 42`
Latent	`latent_type global|local|hybrid`; `vae_latent_dim 32` (3 levels); `latent_ch 4`; `vae_mp_layers 3`; `vae_local_mp_layers 2`; `vae_graph_aware`; `posterior_min_std 0.05`; `z_conditioning adaln`
Loss	`alpha_recon 1000` (MSE); `latent_reg mmd` ($\lambda = 1$) or `kl` (`kl_weight 1e-3`); `beta_aux 10` (peak-to-valley, axial channel); `prior_nll_weight 1`; `prior_grad_to_encoder 0`
Learned distribution	`use_conditional_prior`; `prior_type gnn_e2e`; `prior_family fm`; `prior_mp_layers 3`; `prior_hidden_dim 128`; coarse prior 4 blocks; `prior_fm_steps 30`; `prior_fm_solver heun`; `prior_freeze_epoch 80`
Inference	64 samples per geometry; spread statistic `mag_max` on channels 0–2

Table A.10: Settings shared by the six arms (`configs/MeshGraphNets_Variational/probabilistic/ex3/latent_type/`). Only `latent_type` and `latent_reg` (with its weight) differ.

References

[1] J. W. Hutchinson, W. T. Koiter, Postbuckling theory, Applied Mechanics Reviews 23 (1970) 1353–1366.

[2] S. Lee, H. Cho, H. Kim, S. Shin, Time-domain non-linear aeroelastic analysis via a projection-based reduced-order model, The Aeronautical Journal 124 (1281) (2020) 1798–1818.

[3] S. Lee, K. Jang, H. Cho, H. Kim, S. Shin, Parametric non-intrusive model order reduction for flow-fields using unsupervised machine learning, Computer Methods in Applied Mechanics and Engineering 384 (2021) 113999. doi:10.1016/j.cma.2021.113999.

[4] S. Lee, K. Jang, S. Lee, H. Cho, S. Shin, Parametric model order reduction by machine learning for fluid–structure interaction analysis, Engineering with Computers 40 (2024) 45–60. doi:10.1007/s00366-023-01782-2.

[5] S. Lee, Development of a deep hierarchical variational autoencoder for efficient analysis including fluid–structure interaction, Ph.D. thesis, Seoul National University, Seoul (2024).

[6] S. Lee, S. Lee, K. Jang, H. Cho, S. Shin, Data-driven nonlinear parametric model order reduction framework using deep hierarchical variational autoencoder, Engineering with Computers 40 (2024) 2385–2400. doi:10.1007/s00366-023-01916-6.

[7] S. Lee, S.-H. Kang, S. Lee, S. Shin, Physics-aware neural network-based parametric model-order reduction of the electromagnetic analysis for a coated component, Engineering with Computers 41 (2025) 785–799. doi:10.1007/s00366-024-02056-1.

[8] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, G. E. Dahl, Neural message passing for quantum chemistry, in: Proceedings of the 34th International Conference on Machine Learning, 2017, pp. 1263–1272.

[9] P. W. Battaglia, et al., Relational inductive biases, deep learning, and graph networks, arXiv:1806.01261 (2018).
URL https://arxiv.org/abs/1806.01261

[10] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, P. W. Battaglia, Learning mesh-based simulation with graph networks, in: International Conference on Learning Representations, 2021.
URL https://arxiv.org/abs/2010.03409

[11] M. Fortunato, T. Pfaff, P. Wirnsberger, A. Pritzel, P. Battaglia, MultiScale MeshGraphNets, iCML 2022 AI for Science Workshop; arXiv:2210.00612 (2022).
URL https://arxiv.org/abs/2210.00612

[12] Y. Cao, M. Chai, M. Li, C. Jiang, Efficient learning of mesh-based physical simulation with bi-stride multi-scale graph neural network, in: Proceedings of the 40th International Conference on Machine Learning, Vol. 202 of Proceedings of Machine Learning Research, 2023, pp. 3541–3558.
URL https://proceedings.mlr.press/v202/cao23a.html

[13] S. Lee, D.-H. Park, T. Bang, S.-H. Kang, HI-MeshGraphNets: Efficient and accurate mesh-based physics learning with hierarchical multi-scale graph neural networks, manuscript (2026).

[14] L. Patrignani, S. Pinho, Graph neural networks with hybrid local-global attention for effective prediction of mechanical response in structures, Computer Methods in Applied Mechanics and Engineering 452 (2026) 118753. doi:10.1016/j.cma.2026.118753.

[15] L. Lu, P. Jin, G. Pang, Z. Zhang, G. E. Karniadakis, Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators, Nature Machine Intelligence 3 (2021) 218–229. doi:10.1038/s42256-021-00302-5.

[16] Z. Li, D. Z. Huang, B. Liu, A. Anandkumar, Fourier neural operator with learned deformations for PDEs on general geometries, Journal of Machine Learning Research 24 (388) (2023) 1–26.
URL https://jmlr.org/papers/v24/23-0064.html

[17] M. Lino, T. Pfaff, N. Thuerey, Learning distributions of complex fluid simulations with diffusion graph networks, in: International Conference on Learning Representations, 2025.
URL https://arxiv.org/abs/2504.02843

[18] M. Lino, N. Thuerey, One scale at a time: Scale-autoregressive modeling for fluid flow distributions, arXiv:2604.11403v1 (2026).
URL https://arxiv.org/abs/2604.11403v1

[19] X. Song, T. Yu, Physically-informed flow matching with graph neural networks for complex fluid dynamics, Proceedings of the AAAI Conference on Artificial Intelligence 40 (2) (2026) 1024–1032. doi:10.1609/aaai.v40i2.37072.
URL https://ojs.aaai.org/index.php/AAAI/article/view/37072

[20] M. Lienen, D. Lüdke, J. Hansen-Palmus, S. Günnemann, From zero to turbulence: Generative modeling for 3D flow simulation, in: International Conference on Learning Representations, 2024.
URL https://arxiv.org/abs/2306.01776

[21] R. Molinaro, S. Lanthaler, B. Raonić, et al., Generative AI for fast and accurate statistical computation of fluids, arXiv:2409.18359 (2024).
URL https://arxiv.org/abs/2409.18359

[22] Q. Liu, N. Thuerey, Uncertainty-aware surrogate models for airfoil flow simulations with denoising diffusion probabilistic models, AIAA Journal 62 (8) (2024).
URL https://arxiv.org/abs/2312.05320

[23] I. Price, A. Sanchez-Gonzalez, F. Alet, et al., Probabilistic weather forecasting with machine learning, Nature 637 (2025) 84–90. doi:10.1038/s41586-024-08252-9.

[24] J. Oskarsson, T. Landelius, M. P. Deisenroth, F. Lindsten, Probabilistic weather forecasting with hierarchical graph neural networks, in: Advances in Neural Information Processing Systems, Vol. 37, 2024.
URL https://arxiv.org/abs/2406.04759

[25] F. Alet, I. Price, A. El-Kadi, D. Masters, S. Markou, T. R. Andersson, J. Stott, R. Lam, M. Willson, A. Sanchez-Gonzalez, P. Battaglia, Skillful joint probabilistic weather forecasting from marginals, arXiv:2506.10772 (2025).
URL https://arxiv.org/abs/2506.10772

[26] X. Zhu, B. Sudret, Emulation of stochastic simulators using generalized lambda models, SIAM/ASA Journal on Uncertainty Quantification 9 (4)

(2021) 1345–1380. doi:10.1137/20M1337302.
URL https://arxiv.org/abs/2007.00996

[27] X. Zhu, B. Sudret, Stochastic polynomial chaos expansions to emulate stochastic simulators, International Journal for Uncertainty Quantification 13 (2) (2023) 31–52. doi:10.1615/Int.J.UncertaintyQuantification.2022042912.
URL https://arxiv.org/abs/2202.03344

[28] X. Zhu, M. Broccardo, B. Sudret, Seismic fragility analysis using stochastic polynomial chaos expansions, Probabilistic Engineering Mechanics 72 (2023) 103413. doi:10.1016/j.probengmech.2023.103413.
URL https://arxiv.org/abs/2208.07747

[29] N. Lüthen, S. Marelli, B. Sudret, A spectral surrogate model for stochastic simulators computed from trajectory samples, Computer Methods in Applied Mechanics and Engineering 406 (2023) 115875. doi:10.1016/j.cma.2022.115875.

[30] N. E. Silionis, T. Liangou, K. N. Anyfantis, Deep learning-based surrogate models for spatial field solution reconstruction and uncertainty quantification in structural health monitoring applications, Computers & Structures 301 (2024) 107462. doi:10.1016/j.compstruc.2024.107462.
URL https://www.sciencedirect.com/science/article/abs/pii/S0045794924001913

[31] R. Azimi, Y. Grinberg, D.-X. Xu, O. Liboiron-Ladouceur, Gen-Fab: A variation-aware generative model for predicting fabrication variations in nanophotonic devices, Structural and Multidisciplinary Optimization-ArXiv:2603.11505 (2026).
URL https://arxiv.org/abs/2603.11505

[32] J. Zheng, C. Jahnke, W. Chen, GUST: Quantifying free-form geometric uncertainty of metamaterials using small data, Journal of Mechanical DesignArXiv:2506.12051 (2026).
URL https://arxiv.org/abs/2506.12051

[33] J. Han, B. Bahmani, A multimodal conditional mixture model with distribution-level physics priors, International Journal for Numerical

Methods in Engineering (2026) e70346doi:10.1002/nme.70346.
URL https://arxiv.org/abs/2602.10451

[34] F. Hendriks, O. Rokoš, M. Doškář, M. G. D. Geers, V. Menkovski, Equivariant flow matching for symmetry-breaking bifurcation problems, Machine Learning and the Physical Sciences Workshop, NeurIPS 2025; arXiv:2509.03340 (2025).
URL https://arxiv.org/abs/2509.03340

[35] A. Zimmel, F. Hendriks, M. Holzleitner, F. Sestak, M. Weichselbaumer, V. Menkovski, J. Brandstetter, Bi-FORK: Generative modeling of high-dimensional bifurcating systems, arXiv:2610.12449v1 (2026).
URL https://arxiv.org/abs/2610.12449v1

[36] S. Park, A. Zhou, H. Kim, A. B. Farimani, Generative neural operators through diffusion last layer, arXiv:2602.04139v2; ICML 2026 (2026).
URL https://arxiv.org/abs/2602.04139v2

[37] C. Bülte, P. Scholl, G. Kutyniok, Probabilistic neural operators for functional uncertainty quantification, Transactions on Machine Learning Research (2025).
URL https://arxiv.org/abs/2502.12902

[38] D. P. Kingma, M. Welling, Auto-encoding variational bayes, in: International Conference on Learning Representations, 2014.
URL https://arxiv.org/abs/1312.6114

[39] K. Sohn, H. Lee, X. Yan, Learning structured output representation using deep conditional generative models, in: Advances in Neural Information Processing Systems, Vol. 28, 2015.

[40] Y. Yang, P. Perdikaris, Conditional deep surrogate models for stochastic, high-dimensional, and multi-fidelity systems, Computational Mechanics 64 (2019) 417–434. doi:10.1007/s00466-019-01718-y.
URL https://arxiv.org/abs/1901.04878

[41] X. Shen, N. Meinshausen, Engression: extrapolation through the lens of distributional regression, Journal of the Royal Statistical Society Series B: Statistical Methodology (2025).
URL https://arxiv.org/abs/2307.00835

[42] L. Pacchiardi, R. A. Adewoye, R. Dutta, Probabilistic forecasting with generative networks via scoring rule minimization, Journal of Machine Learning Research 25 (45) (2024) 1–64.
URL https://arxiv.org/abs/2112.08217

[43] K. Preechakul, N. Chatthee, S. Wizadwongsa, S. Suwajanakorn, Diffusion autoencoders: Toward a meaningful and decodable representation, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022.
URL https://arxiv.org/abs/2111.15640

[44] P. Dahlinger, N. Freymuth, M. Volpp, T. Hoang, G. Neumann, Latent task-specific graph network simulators, arXiv:2311.05256 (2023).
URL https://arxiv.org/abs/2311.05256

[45] B. Nath, M. Lakhadive, A. Sharma, B. Bhowmik, A physics-aware variational graph autoencoder for joint modal identification with uncertainty quantification, arXiv:2604.11237 (2026).
URL https://arxiv.org/abs/2604.11237

[46] T. Park, M.-Y. Liu, T.-C. Wang, J.-Y. Zhu, Semantic image synthesis with spatially-adaptive normalization, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019.
URL https://arxiv.org/abs/1903.07291

[47] P. Zhu, R. Abdal, Y. Qin, P. Wonka, SEAN: Image synthesis with semantic region-adaptive normalization, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 5104–5113.
URL https://arxiv.org/abs/1911.12861

[48] Z. Tan, M. Chai, D. Chen, J. Liao, Q. Chu, B. Liu, G. Hua, N. Yu, Diverse semantic image synthesis via probability distribution modeling, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2021, pp. 7962–7971.
URL https://arxiv.org/abs/2103.06878

[49] C. F. Baumgartner, K. C. Tezcan, K. Chaitanya, A. M. Hötker, U. J. Muehlematter, K. Schawkat, A. S. Becker, O. Donati, E. Konukoglu, PHiSeg: Capturing uncertainty in medical image segmentation, in:

Medical Image Computing and Computer Assisted Intervention (MICCAI), 2019.
URL https://arxiv.org/abs/1906.04045

[50] S. A. A. Kohl, B. Romera-Paredes, K. H. Maier-Hein, D. J. Rezende, S. M. A. Eslami, P. Kohli, A. Zisserman, O. Ronneberger, A hierarchical probabilistic U-Net for modeling multi-scale ambiguities, arXiv:1905.13077 (2019).
URL https://arxiv.org/abs/1905.13077

[51] X. Zeng, A. Vahdat, F. Williams, Z. Gojcic, O. Litany, S. Fidler, K. Kreis, LION: Latent point diffusion models for 3d shape generation, in: Advances in Neural Information Processing Systems, Vol. 35, 2022.
URL https://arxiv.org/abs/2210.06978

[52] G. Yang, X. Huang, Z. Hao, M.-Y. Liu, S. Belongie, B. Hariharan, PointFlow: 3d point cloud generation with continuous normalizing flows, in: IEEE/CVF International Conference on Computer Vision, 2019.
URL https://arxiv.org/abs/1906.12320

[53] Y. Xie, S. J. Pan, Global and local topology-aware graph generation via dual conditioning diffusion, in: International Conference on Learning Representations, 2026.
URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/af2bb2b2280d36f8842e440b4e275152-Abstract-Conference.html

[54] K. Masuki, Y. Ashida, Renormalization group flow matching for scalable local generative modeling, arXiv:2608.23696 (2026).
URL https://arxiv.org/abs/2608.23696

[55] T. Karras, S. Laine, T. Aila, A style-based generator architecture for generative adversarial networks, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019.
URL https://arxiv.org/abs/1812.04948

[56] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, A. Lerchner, β-VAE: Learning basic visual concepts with a constrained variational framework, in: International Conference on Learning Representations, 2017.

[57] S. Zhao, J. Song, S. Ermon, InfoVAE: Balancing learning and inference in variational autoencoders, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 33, 2019, pp. 5885–5892.
URL https://arxiv.org/abs/1706.02262

[58] I. Tolstikhin, O. Bousquet, S. Gelly, B. Schölkopf, Wasserstein auto-encoders, in: International Conference on Learning Representations, 2018.
URL https://arxiv.org/abs/1711.01558

[59] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, A. Smola, A kernel two-sample test, Journal of Machine Learning Research 13 (2012) 723–773.

[60] P. Ghosh, M. S. M. Sajjadi, A. Vergari, M. Black, B. Schölkopf, From variational to deterministic autoencoders, in: International Conference on Learning Representations, 2020.
URL https://arxiv.org/abs/1903.12436

[61] B. Dai, D. Wipf, Diagnosing and enhancing VAE models, in: International Conference on Learning Representations, 2019.
URL https://arxiv.org/abs/1903.05789

[62] A. Vahdat, K. Kreis, J. Kautz, Score-based generative modeling in latent space, in: Advances in Neural Information Processing Systems, Vol. 34, 2021.
URL https://arxiv.org/abs/2106.05931

[63] T. Gneiting, A. E. Raftery, Strictly proper scoring rules, prediction, and estimation, Journal of the American Statistical Association 102 (477) (2007) 359–378.

[64] M. Scheuerer, T. M. Hamill, Variogram-based proper scoring rules for probabilistic forecasts of multivariate quantities, Monthly Weather Review 143 (4) (2015) 1321–1334.

[65] R. Pic, C. Dombry, P. Naveau, M. Taillardat, Proper scoring rules for multivariate probabilistic forecasts based on aggregation and transformation, arXiv:2407.00650 (2024).
URL https://arxiv.org/abs/2407.00650

[66] T. M. Hamill, Interpretation of rank histograms for verifying ensemble forecasts, Monthly Weather Review 129 (3) (2001) 550–560.

[67] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, T. Aila, Improved precision and recall metric for assessing generative models, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.

[68] M. F. Naeem, S. J. Oh, Y. Uh, Y. Choi, J. Yoo, Reliable fidelity and diversity metrics for generative models, in: Proceedings of the 37th International Conference on Machine Learning, 2020.
URL https://arxiv.org/abs/2002.09797

[69] R. Kansal, A. Li, J. Duarte, N. Chernyavskaya, M. Pierini, B. Orzari, T. Tomei, Evaluating generative models in high energy physics, Physical Review D 107 (2023) 076017.
URL https://arxiv.org/abs/2211.10295

[70] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville, FiLM: Visual reasoning with a general conditioning layer, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 32, 2018.
URL https://arxiv.org/abs/1709.07871

[71] W. Peebles, S. Xie, Scalable diffusion models with transformers, in: IEEE/CVF International Conference on Computer Vision, 2023.
URL https://arxiv.org/abs/2212.09748

[72] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le, Flow matching for generative modeling, in: International Conference on Learning Representations, 2023.
URL https://arxiv.org/abs/2210.02747

[73] X. Liu, C. Gong, Q. Liu, Flow straight and fast: Learning to generate and transfer data with rectified flow, in: International Conference on Learning Representations, 2023.
URL https://arxiv.org/abs/2209.03003

[74] T. Karras, M. Aittala, T. Aila, S. Laine, Elucidating the design space of diffusion-based generative models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022.
URL https://arxiv.org/abs/2206.00364